



Online Grocery Store MVP — Final Stack Note

Model: Single store (own inventory) **Fulfillment:** Delivery + Store Pickup **Platforms:** iOS + Android + Web (Cross-Platform, one codebase)

Frontend

Layer	Technology
Framework	Expo (React Native + React Native Web) — Universal App
Styling	NativeWind (Tailwind for React Native)
State Management	Zustand
Navigation	Expo Router

One codebase → builds to iOS, Android, and Web (via React Native Web).

Backend

Layer	Technology
Framework	Node.js + Express.js
Language	JavaScript / TypeScript
API Type	REST API

One single backend deployment — serves mobile app + web app both. No separate backend per platform.

Database & Storage

Layer	Technology
Database	Supabase (PostgreSQL)
ORM	Prisma
Auth	Supabase Auth
File/Image Storage	Supabase Storage
Caching (optional, add later when traffic grows)	Redis (via Upstash)

Core Tables: `products` (unit, stock_quantity, category), `orders` (fulfillment_type: delivery/pickup, delivery_slot, pickup_time), `order_items`, `store_info`, `user_interactions`, `style_profiles`, `chat_conversations`, `profiles`

AI Agents (Key Differentiator)

Agent	Purpose
Smart Reorder Agent	Detects repeat purchase patterns (e.g. weekly milk/eggs), proactively suggests reorder
Shopping Assistant Agent	Chat-based product search, recipe-to-cart conversion, adds items to cart via function/tool calling

Engine: Claude API (with function/tool calling) **Tools used by agent:** `get_style_profile()`, `search_products()`, `add_to_cart()`, `get_product_details()`

Payments

Layer	Technology
Gateway	Stripe
Local option	JazzCash / EasyPaisa
Extra	Cash on Delivery / Cash on Pickup

🌫 Hosting & Deployment

Component	Where	Notes
Backend (Node.js API)	Railway (or Render)	✅ Single deployment — shared by mobile + web
Database	Supabase (managed)	Single instance
Web App	Vercel	Deploy via <code>npx expo export --platform web</code> → static files → Vercel
iOS App	Build only, no store submission	<code>eas build --platform ios --profile preview</code> → <code>.ipa</code> file for direct/simulator testing
Android App	Build only, no store submission	<code>eas build --platform android --profile preview</code> → <code>.apk</code> file, directly installable (sideload)
Mobile Build Service	Expo EAS	Cloud build service — no local Mac/Xcode needed

⚠ **App Store / Play Store submission skipped for now** — `eas submit` step is not used.
Builds are for internal testing / sideloading only.

Notes on installing builds without a store:

- **Android:** `.apk` file installs directly on any Android phone (enable "install from unknown sources").
- **iOS:** Apple restricts installing on real devices without a store — either test on the **Simulator**, or later use **TestFlight** (still requires Apple Developer account, \$99/year, but skips full App Store review).

Developer account costs (only needed later, when ready to publish):

- Google Play Console: \$25 (one-time)
- Apple Developer Program: \$99/year

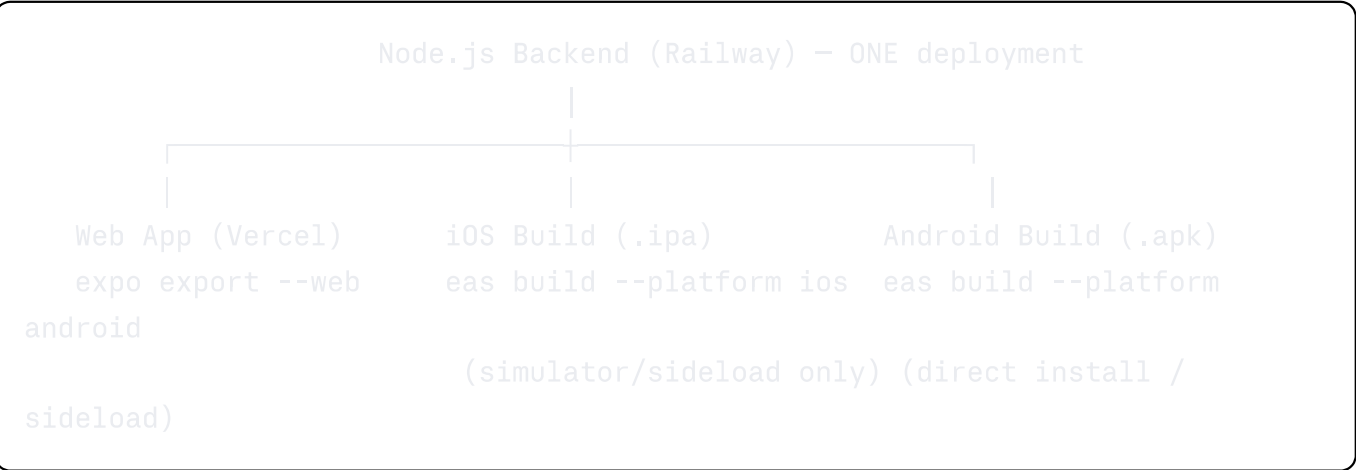
 Extra Tools

Purpose	Tool
Error Tracking	Sentry
Analytics	Google Analytics / Mixpanel
Email (order confirmations)	Resend
Search (MVP stage)	Postgres full-text search

 One-Glance Stack Summary

Frontend:	Expo (React Native + RN Web) + NativeWind + Zustand + Expo Router
Backend:	Node.js + Express → deployed once on Railway
Database:	Supabase (PostgreSQL) + Prisma ORM
Auth:	Supabase Auth
Caching:	Redis (Upstash) – optional, add later
AI Agents:	Claude API – Smart Reorder Agent + Shopping Assistant Agent
Payments:	Stripe + JazzCash/EasyPaisa + Cash on Delivery/Pickup
Storage:	Supabase Storage
Web Hosting:	Vercel
Mobile Build:	Expo EAS (build only – .apk / .ipa, no store submission yet)
Monitoring:	Sentry

 Deployment Flow (Quick Reference)



Key rule to remember:

- Backend + Database = **one place**, shared by everything

- Web frontend = **Vercel**
- Mobile frontend = **build only via Expo EAS** for now — installable files, no App Store / Play Store publishing yet